

Introduction

Definition of Doubly Linked List

- Doubly linked lists are a linear data structure where each node contains data and references to both the next and the previous nodes.

Overview of Linked List Structure

- Nodes contain data, a reference to the next node, and a reference to the previous node, forming a doubly linked structure.

Purpose and Common Use Cases

- Doubly linked lists provide bidirectional traversal.
 - Useful in scenarios where both forward and backward traversal is necessary.
-

Characteristics of a Doubly Linked List

- ✓ Bidirectional Traversal
 - ✓ Insertion and Deletion Flexibility
 - ✓ Head and Tail Pointers
 - ✓ Reversibility
 - ✓ Memory Overhead
-

Why Use Doubly Linked Lists?

Advantages over Singly Linked Lists

- Bidirectional traversal
- Easier removal of nodes
- More efficient backward navigation

Flexibility in Memory Usage

- Similar to singly linked lists
- Provide flexibility in memory utilization

Applications

- Useful when both forward and backward traversal is required
-

Linked List Implementation (in C)

Overview

- Similar to singly linked lists, but nodes now have two pointers:
 - One for the next node
 - One for the previous node

Representation of Nodes

- Nodes consist of:
 - Data field
 - Pointer to next node
 - Pointer to previous node

Structure in C

```
struct node {  
    int data;  
    struct node *next;  
    struct node *prev;  
};
```

Head and Tail Pointers

- Used for efficient traversal in both directions
-

Basic Linked List Operations

Insertion

- insertFront and insertEnd
- Update both next and previous pointers

Deletion

- deleteFront and deleteEnd
- Adjust both next and previous pointers

Search

- Searching is bidirectional (forward and backward)
-

Memory Management in Doubly Linked Lists

Dynamic Memory Allocation

- Nodes are dynamically allocated using `malloc`
- Memory is deallocated using `free` to prevent memory leaks

Importance of Freeing Memory

- Crucial to free allocated memory for both next and previous pointers
-

Basic Operations on Doubly Linked List

- **Insertion** – Adds an element at the beginning of the list
- **Deletion** – Deletes an element at the beginning
- **Insert Last** – Adds an element at the end
- **Delete Last** – Deletes an element from the end
- **Insert After** – Adds an element after a specific item
- **Delete** – Deletes an element using a key
- **Display Forward** – Displays list in forward direction
- **Display Backward** – Displays list in backward direction.

Structure of Doubly Linked List (DLL)

C Structure

```
struct node
{
    int data;

    node *next;

    node *previous; // holds the address of previous node
};
```

Node Representation

- Each node contains:
 - Previous pointer (prev) → points to previous node
 - Data field → stores value
 - Next pointer (next) → points to next node

Insertion in Doubly Linked List

1. Insertion at Beginning

Steps:

- Create a new node (**ptr**)
- Set:
 - **ptr->prev = NULL**
 - **ptr->next = head**
- If list is not empty:
 - **head->prev = ptr**

- Update:
 - `head = ptr`
-

2. Insertion After a Given Node

Steps:

- Let the given node be `temp`
 - Create new node `ptr`
 - Set:
 - `ptr->next = temp->next`
 - `ptr->prev = temp`
 - If next node exists:
 - `temp->next->prev = ptr`
 - Update:
 - `temp->next = ptr`
-

3. Insertion at End

Steps:

- Traverse to last node
 - Create new node `ptr`
 - Set:
 - `ptr->next = NULL`
 - `ptr->prev = last node`
 - Update:
 - `last node->next = ptr`
-

Deletion in Doubly Linked List

Deletion can occur:

- At the beginning

- At the end
 - At a specific position
-

1. Deletion at Beginning

Steps:

- Store head in temporary node
 - Move head to next node:
 - `head = head->next`
 - Set:
 - `head->prev = NULL`
 - Delete temporary node
-

2. Deletion at a Given Position

Steps:

- Traverse to the node to be deleted (`temp`)
 - Update links:
 - `temp->prev->next = temp->next`
 - `temp->next->prev = temp->prev`
 - Delete `temp`
-

3. Deletion at End

Steps:

- Traverse to last node
 - Store it in temporary node
 - Move to second last node
 - Set:
 - `secondLast->next = NULL`
 - Delete temporary node
-

Time Complexity of Doubly Linked List

Operation	Complexity
Access / Traversal	$O(n)$
Insertion at Beginning	$O(1)$
Insertion at End (with tail)	$O(1)$
Insertion at End (without tail)	$O(n)$
Insertion/Deletion in Middle	$O(n/2) \approx O(n)$
Deletion at Beginning	$O(1)$
Deletion at End (with tail)	$O(1)$
Deletion at End (without tail)	$O(n)$
Searching for an Element	$O(n)$
Reversal	$O(n)$

👉 Key Idea:

- Operations are fast ($O(1)$) when position is known
 - Slow ($O(n)$) when traversal is required
-

Space Complexity

Component	Complexity
Node Storage	$O(n)$
Additional Pointers (head, tail)	$O(1)$
Total Space Complexity	$O(n)$

👉 Each node stores:

- Data
- Next pointer
- Previous pointer → increases memory usage compared to singly linked list

Circular Doubly Linked List

Explanation

- Similar to circular singly linked list
- Each node points to:
 - **Next node**
 - **Previous node**
- Last node connects back to the first node

Benefits & Applications

- Efficient traversal in loops
- No NULL pointers
- Useful in:
 - Music playlist
 - Undo/Redo operations
 - Round-robin scheduling

Advantages of Doubly Linked Lists

- ✓ **Bidirectional Traversal**
- ✓ **Easier Removal of Nodes**
- ✓ **More Efficient Backward Navigation**

Common Mistakes / Pitfalls

✗ Forgetting to update both pointers

- Must update:
 - `next`
 - `prev`

✗ Incorrect handling of edge cases

- Empty list
- Single node
- First/last node

✗ Memory leaks

- Not using `free()` properly
- Both pointers must be handled before deletion

All operations of DLL:-

```
C/C++
```

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct node {
    int data;
    struct node *prev;
    struct node *next;
};
```

```
struct node *head = NULL;
```

```
// Create Node
```

```
struct node* createNode(int data) {
    struct node *newNode = (struct node*)malloc(sizeof(struct
node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;
    return newNode;
}
```

```
// Insert at Beginning
```

```

void insertFront(int data) {
    struct node *newNode = createNode(data);
    if (head == NULL) {
        head = newNode;
        return;
    }
    newNode->next = head;
    head->prev = newNode;
    head = newNode;
}

```

// Insert at End

```

void insertEnd(int data) {
    struct node *newNode = createNode(data);
    if (head == NULL) {
        head = newNode;
        return;
    }
    struct node *temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
    newNode->prev = temp;
}

```

// Insert After Given Value

```

void insertAfter(int key, int data) {
    struct node *temp = head;
    while (temp != NULL && temp->data != key)
        temp = temp->next;
    if (temp == NULL) {
        printf("Value not found!\n");
        return;
    }
    struct node *newNode = createNode(data);
    newNode->next = temp->next;

```

```

    newNode->prev = temp;
    if (temp->next != NULL)
        temp->next->prev = newNode;
    temp->next = newNode;
}

```

// Delete at Beginning

```

void deleteFront() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct node *temp = head;
    head = head->next;
    if (head != NULL)
        head->prev = NULL;
    free(temp);
}

```

// Delete at End

```

void deleteEnd() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct node *temp = head;
    if (temp->next == NULL) {
        head = NULL;
        free(temp);
        return;
    }
    while (temp->next != NULL)
        temp = temp->next;
    temp->prev->next = NULL;
    free(temp);
}

```

// Delete by Value

```
void deleteValue(int key) {
    struct node *temp = head;
    while (temp != NULL && temp->data != key)
        temp = temp->next;
    if (temp == NULL) {
        printf("Value not found!\n");
        return;
    }
    if (temp == head) {
        deleteFront();
        return;
    }
    if (temp->next == NULL) {
        deleteEnd();
        return;
    }
    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;
    free(temp);
}
```

// Display Forward

```
void displayForward() {
    struct node *temp = head;
    if (temp == NULL) {
        printf("List is empty\n");
        return;
    }
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

```

// Display Backward
void displayBackward() {
    struct node *temp = head;
    if (temp == NULL) {
        printf("List is empty\n");
        return;
    }
    while (temp->next != NULL)
        temp = temp->next;
    while (temp != NULL) {
        printf("%d <--> ", temp->data);
        temp = temp->prev;
    }
    printf("NULL\n");
}

int main() {
    int choice, data, key;
    while (1) {
        printf("\n--- Doubly Linked List ---\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at End\n");
        printf("3. Insert After\n");
        printf("4. Delete at Beginning\n");
        printf("5. Delete at End\n");
        printf("6. Delete by Value\n");
        printf("7. Display Forward\n");
        printf("8. Display Backward\n");
        printf("9. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter data: ");
                scanf("%d", &data);

```

```
        insertFront(data);
        break;
    case 2:
        printf("Enter data: ");
        scanf("%d", &data);
        insertEnd(data);
        break;
    case 3:
        printf("Enter key value: ");
        scanf("%d", &key);
        printf("Enter data: ");
        scanf("%d", &data);
        insertAfter(key, data);
        break;
    case 4:
        deleteFront();
        break;
    case 5:
        deleteEnd();
        break;
    case 6:
        printf("Enter value to delete: ");
        scanf("%d", &key);
        deleteValue(key);
        break;
    case 7:
        displayForward();
        break;
    case 8:
        displayBackward();
        break;
    case 9:
        exit(0);
    default:
        printf("Invalid choice!\n");
}
```

```
}  
return 0;  
}
```